

COMS3011A Lab 2

Brendan Griffiths

1 Sustainable, Repeatable Infrastructure

A common excuse in software engineering is the mythical *it works on my machine*. It occurs as a result of minor changes in the execution environment of a given service, and is painful to debug, because reproducing the problem needs access to the production environment.

The solution is a technique called **Containerisation**. A service declaratively specifies what software and versions it relies on and is executed in a self-contained environment with just the installed dependencies. Each service (application, api, database etc) runs in its own container and relies on specified dependencies. Then, these services can be organised into a multi-container application that describes what *images* (the files and programs used in a running container) are needed, how each service connects to each other, and their configuration.

The most popular containerisation platform is Docker with most cloud providers offering products that directly interface with Docker based architectures ([Google Cloud](#), [AWS](#), [Azure](#), [Cloudflare](#)). You can read more about Docker's concepts on their [overview](#), and a [containerisation guide](#).

As part of this lab, you need to containerise the provided application, and configure a multi-container architecture for it.

1.1 Application

The provided application is a SvelteKit full stack Guestbook App that allows users to leave comments and reviews. It is built with Vite, served by Node, and it connects to a PostgreSQL database. The app creates its own `entries` table on startup, so you never need to run SQL by hand.

1.1.1 Configuration

The app is configured through the following environment variables:

Variable	Required	Default	Purpose
DATABASE_URL	yes		postgres://USER:PASSWORD@HOST:5432/DBNAME
PORT	no	3000	Port the server listens on
HOST	no	0.0.0.0	Interface the server binds to

1.1.2 Building and running

```
npm ci           # install exact dependencies from package-lock.json
npm run build    # compile into build/
npm prune --omit=dev # optional: drop build-only dependencies
node build       # start the server (listens on $PORT, default 3000)
```

At runtime the server only needs the `build/` directory, the production `node_modules` and `package.json`. None of the source code or dev dependencies are needed.

1.1.3 Supported versions

- **Node:** 20.19+, 22.12+ or 24+. `.npmrc` sets `engine-strict=true`, so `npm ci` fails on any other version.
- **PostgreSQL:** 14 or newer, using the official postgres image.

1.1.4 Diagnostic endpoint

`GET /api/marking-test` returns JSON describing the running API and its database connection. It returns HTTP 200 with `"status": "ok"` when everything works, and HTTP 503 with `"status": "error"` when the database can't be reached. It is also a useful endpoint to healthcheck against.

1.2 Requirements

You may choose the service names, database name, credentials, volume names and host port. Your application must:

1. Define **two services**:
 - The app built from its `Dockerfile`
 - The official postgres image.
2. Give the app a `DATABASE_URL` environment variable of the form `postgres://USER:PASSWORD@HOST:5432/DBNAME`, and set `NODE_ENV=production` on the app container.
3. Publish the app's port to the host so the site is reachable in a browser.
4. Keep the guestbook's data when the stack is stopped and started again with `docker compose down / docker compose up`.
5. Start reliably: the app should not come up before the database is ready to accept connections.

1.3 Submission

You must submit two files for the lab:

1. `Dockerfile`
2. `compose.yml` (or `compose.yaml` / `docker-compose.yml` / `docker-compose.yaml`)

Put both in the root of the repository. They may live in a subdirectory instead, as long as your compose file's `build.context` and `build.dockerfile` point at the app and your `Dockerfile`.

1.4 Marking

Automarked but **not via moodle's automarker**.

I have provided a test script called `automark.sh` that builds the container image, starts the compose, and then checks the running stack through a series of stages. Each of the 28 required checks is worth the same amount, so your mark is the number of checks passed out of 28.

Stage	What is checked	Checks	Weight
1. Static checks	Compose file exists and is valid; exactly two services; app service has a <code>build:</code> key; Dockerfile exists; database service runs a postgres image; app publishes a port to the host	7	25.0%
2. Build and start	Images build; <code>docker compose up -d</code> succeeds; <code>/api/marking-test</code> reports ok within 90s	3	10.7%
3. Marking-test endpoint	Status ok; API runs in a container; <code>NODE_ENV=production</code> ; database connected; postgres version 14 or later; entries table exists; database accepts writes; database host is not localhost; app uses the database container	9	32.1%
4. App smoke test	Form submission succeeds; new entry appears on the home page; entry count increases by one	3	10.7%
5. Persistence	Stack comes back after <code>docker compose down / up</code> ; entry count preserved; entry still listed	3	10.7%
6. Full-stack restart	Marking-test ok after <code>docker compose restart</code> ; data intact	2	7.1%
7. Database-only restart	API reconnects after the database service restarts	1	3.6%
Total		28	100%

The script also runs some “good practice” checks (non-root user, small image, no credentials baked into the image, `depends_on`, database healthcheck, unpublished database port, named volume, `.dockerignore`). These only produce warnings and do not affect your mark.

1.5 Running the automarker

`automark.sh` builds and starts your stack, tests it and tears it down again (`docker compose down -v`, unless `--keep` is passed). It needs Docker with Compose v2, `curl` and `jq`, and the host port your compose file publishes must be free. It finds your app service as the one with a `build:` key and your database service as the one running a postgres image, so your naming choices don’t matter.

```
./automark.sh           # mark the Dockerfile + compose file in this directory
./automark.sh --keep    # leave the stack running afterwards so you can explore it
./automark.sh path/to/dir # mark a submission in another directory
```

The `PROJECT` (compose project name, default `automark`), `TIMEOUT` (seconds to wait for the app, default 90) and `BASE_URL` (skip port discovery) environment variables can be overridden.

If something essential fails (a missing published port, a failed build, the stack not starting, or the app never becoming ready) the script stops early and prints the last 40 lines of container logs. Checks that never ran count as not passed. If the database service can’t be identified as a postgres image, the 3 checks that depend on it are skipped and left out of the total. The script exits with status 0 only if every required check passed.

AI Declaration: Review, Generation - Claude Code (Sonnet 5.5)